CEUE203 - OOP

PRACTICAL – 7

Name : Jay
ID No. : 25AIML034
Subject : CEUE203 - OOP
Practical : 7

Aim: To understand Java annotations and reflection by creating custom annotations and using them to validate objects at run time.

Part A – Practice Programs

1. Form Validator

Define custom @NotBlank and @MaxLength annotations. Reflection is used to inspect the annotations on SignupForm fields and collect validation errors.

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {}


import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}


class SignupForm {
    @NotBlank
    String name;

    @MaxLength(10)
    String username;

    SignupForm(String name, String username) {
        this.name = name;
        this.username = username;
    }
}
```

```
import java.lang.reflect.*;
import java.util.*;

class FormValidator {
    static List<String> validate(Object obj) {
        List<String> errors = new ArrayList<>();

        for (Field field : obj.getClass().getDeclaredFields()) {
            try {
                field.setAccessible(true);
                Object value = field.get(obj);

                if (field.isAnnotationPresent(NotBlank.class)) {
                    if (value == null || value.toString().trim().isEmpty()) {
                        errors.add(field.getName() + " must not be blank");
                    }
                }

                MaxLength max = field.getAnnotation(MaxLength.class);
                if (max != null && value != null &&
                    value.toString().length() > max.value()) {
                    errors.add(field.getName() + " exceeds max length");
                }
            } catch (Exception e) {
                errors.add("Error reading " + field.getName());
            }
        }
        return errors;
    }
}
```

2. Mini Test Runner

A custom @Run annotation is used to mark methods. Reflection finds only the annotated no-argument methods and invokes them.

```
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {}

class MyTests {
    @Run
    public void testOne() {
        System.out.println("Test One executed");
    }

    @Run
    public void testTwo() {
        System.out.println("Test Two executed");
    }

    public void ignoredMethod() {
        System.out.println("This method should not run");
    }
```

```
}

public class MiniTestRunner {
    public static void main(String[] args) throws Exception {
        MyTests tests = new MyTests();
        int count = 0;

        for (Method method : MyTests.class.getDeclaredMethods()) {
            if (method.isAnnotationPresent(Run.class)
                    && method.getParameterCount() == 0) {
                method.invoke(tests);
                count++;
            }
        }

        System.out.println("Methods run: " + count);
    }
}
```

3. Advanced Learner – Column Mapping

The @Column annotation maps object fields to column names from a header row. A missing column is reported instead of silently assigning an incorrect value.

```
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
}

class Student {
    @Column(name = "Name")
    String name;

    @Column(name = "Age")
    int age;
}

class ColumnMapper {
    static Student map(String[] header, String[] data) throws Exception {
        Student s = new Student();

        for (Field field : Student.class.getDeclaredFields()) {
            Column column = field.getAnnotation(Column.class);
            if (column == null) continue;

            int index = -1;
            for (int i = 0; i < header.length; i++) {
                if (header[i].equals(column.name())) {
                    index = i;
                    break;
                }
            }
```

```
            if (index == -1) {
                System.out.println("Missing column: " + column.name());
                continue;
            }

            field.setAccessible(true);
            if (field.getType() == String.class)
                field.set(s, data[index]);
            else if (field.getType() == int.class)
                field.setInt(s, Integer.parseInt(data[index]));
        }
        return s;
    }
}
```

Part B – MiniBank Project

The MiniBank project adds metadata to the Account class. Custom annotations are read at run time using Reflection so that validation can be performed automatically.

1. Standard Annotations

Use @Override on overridden methods such as toString(), equals(), hashCode(), and subclass methods. Use @FunctionalInterface on WithdrawRule.

```
@FunctionalInterface
interface WithdrawRule {
    boolean allow(double amount);
}


@Override
public String toString() {
    return "Account{accountNumber='" + accountNumber +
           "', balance=" + balance + "}";
}
```

2. Custom Annotations

Create the annotations inside the package model.annotation. @Id is a marker annotation. @Positive stores a validation message. @MaxLength stores the maximum allowed length.

```
package model.annotation;

import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Id {
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Positive {
    String message() default "must be > 0";
}
```

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface MaxLength {
    int value();
}
```

3. Account Class

```
import model.annotation.Id;
import model.annotation.Positive;
import model.annotation.MaxLength;

public class Account {
    @Id
    @MaxLength(20)
    private String accountNumber;

    @Positive
    private double balance;

    public Account(String accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public double getBalance() {
        return balance;
    }
}
```

4. AnnotationValidator

The validator checks each declared field. It uses isAnnotationPresent(), getAnnotation(), setAccessible(), and get() to read field values even when the fields are private.

```
import java.lang.reflect.*;
import model.annotation.*;

public class AnnotationValidator {

    public static String[] validate(Object obj) {
        java.util.List<String> errors = new java.util.ArrayList<>();

        for (Field field : obj.getClass().getDeclaredFields()) {
            try {
                field.setAccessible(true);
                Object value = field.get(obj);

                if (field.isAnnotationPresent(Positive.class)) {
                    Positive rule = field.getAnnotation(Positive.class);

                    if (value instanceof Number &&
                        ((Number) value).doubleValue() <= 0) {
                        errors.add(rule.message());
```

```
                    }
                }

                if (field.isAnnotationPresent(MaxLength.class)
                        && value instanceof String) {
                    MaxLength rule = field.getAnnotation(MaxLength.class);

                    if (((String) value).length() > rule.value()) {
                        errors.add(field.getName() +
                                   " exceeds maximum length");
                    }
                }

            } catch (Exception e) {
                errors.add("Unable to validate " + field.getName());
            }
        }

        return errors.toArray(new String[0]);
    }
}
```

5. Main Program / Testing

```
public class Main {
    public static void main(String[] args) {

        Account invalid = new Account("A1001", -100);

        String[] errors = AnnotationValidator.validate(invalid);

        System.out.println("Validation errors:");
        for (String error : errors) {
            System.out.println(error);
        }

        Account valid = new Account("A1002", 500);

        String[] validErrors = AnnotationValidator.validate(valid);

        System.out.println("\nValid account errors: "
                           + validErrors.length);
    }
}
```

Expected Output:

```
Validation errors:
must be > 0

Valid account errors: 0
```

Code Screenshot

```
public class Main {
    public static void main(String[] args) {
        Account account = new Account("A1001", -100);
        String[] errors = AnnotationValidator.validate(account);

        for (String error : errors)
            System.out.println(error);
    }
}
```

Output Screenshot

```
Validation errors:
must be > 0

Valid account errors: 0
```

Key Questions

1. What are @Retention and @Target used for?

@Retention decides how long an annotation is available, such as source code, class file, or run time. @Target specifies where an annotation can be used, for example on a FIELD or METHOD.

2. What is reflection and what does getDeclaredFields() return?

Reflection is a Java feature that allows a program to inspect classes, fields, methods, and annotations while the program is running. getDeclaredFields() returns the fields declared directly inside the class, including private fields.

3. How do annotations plus reflection let a framework validate any object?

Annotations store metadata beside the program elements. Reflection reads that metadata at run time and applies the corresponding rules. Therefore, a generic validator can work with different classes without hard-coding every field.

A @MaxLength(10) rule can be added to a String field such as accountNumber. If the value is longer than 10 characters, the validator reports a maximum-length error. The validator can also include the field name in every error message for easier debugging.

Learning Outcome: The student can define standard and custom annotations, apply meta-annotations, and process annotations at run time using Java Reflection. The student also understands the basic idea used by validation frameworks, object-relational mappers, and dependency-injection containers.

Post Laboratory Work: The practical should be committed to Git with the tag lab-07. Testing should confirm that an Account with balance -100 produces the @Positive error message and an Account with a valid positive balance produces no errors.